Design and Optimization of an Urdu Automatic Speech Recognition System for Emergency Response

---

Model Architecture and Design Choices

The system adopts a transformer-based encoder-decoder architecture inspired by Whisper-like models, primarily due to their strong performance in multilingual and noisy environments. The encoder processes log-Mel spectrogram representations of the input signal, extracting hierarchical acoustic features, while the decoder generates token sequences conditioned on both the encoded audio features and previously generated tokens.

A critical design decision involved selecting between a purely end-to-end model and a hybrid system incorporating language models. While end-to-end models simplify deployment, incorporating a domain-adapted language model during decoding significantly improves performance in recognizing emergency-specific vocabulary such as “آگ” (fire), “حادثہ” (accident), and “مدد” (help). Beam search decoding with language model fusion was therefore chosen to balance accuracy and inference complexity.

Data Strategy and Preprocessing Pipeline

The effectiveness of the ASR system is heavily dependent on the quality and diversity of training data. Given the scarcity of large-scale Urdu emergency call datasets, a hybrid data strategy was adopted. Public datasets such as Mozilla Common Voice provide a baseline for general speech recognition, but they lack the acoustic and linguistic characteristics of emergency scenarios. To address this gap, synthetic data generation techniques were employed, including scripted emergency dialogues recorded under simulated noisy conditions.

Preprocessing plays a crucial role in normalizing input data. Audio signals are resampled to 16kHz and passed through a Voice Activity Detection (VAD) module to remove silence and irrelevant segments. Noise reduction is applied using spectral subtraction methods, while segmentation ensures that input sequences remain within manageable lengths for transformer processing. Data augmentation techniques, including background noise injection and speed perturbation, are applied to improve generalization.

Training Methodology and Optimization

Training the ASR model involves fine-tuning a pretrained multilingual model on domain-specific data. The loss function is defined as cross-entropy over token predictions, with label smoothing applied to prevent overconfidence in predictions. Optimization is performed using the AdamW optimizer with a carefully tuned learning rate schedule to avoid catastrophic forgetting of pretrained knowledge.

One of the critical considerations during training is balancing between general language understanding and domain specialization. Overfitting to emergency-specific vocabulary can degrade performance on general speech patterns, so a mixed training strategy is employed, combining general Urdu datasets with domain-specific samples in a weighted manner.

Evaluation and Error Analysis

Evaluation of the ASR system is conducted using standard metrics such as Word Error Rate (WER) and Character Error Rate (CER). However, these metrics are supplemented with domain-specific evaluations focusing on keyword recall. For instance, failing to detect a word like “آگ” (fire) is significantly more critical than minor grammatical errors.

Error analysis reveals that the system struggles primarily with code-switching scenarios where speakers mix Urdu and English, as well as with heavy background noise. Overlapping speech remains a challenging area, suggesting the potential need for speaker diarization in future iterations.

Deployment Considerations

The ASR model is deployed as a microservice within the backend architecture, exposed via RESTful APIs. Given the computational demands of transformer models, GPU acceleration is utilized for inference. To ensure scalability, the system is containerized using Docker and deployed behind a load balancer, allowing multiple instances to handle concurrent requests.

Latency optimization is achieved through model quantization and batching strategies, ensuring that the system meets real-time processing requirements without significant degradation in accuracy.

Integration with Downstream Systems

The output of the ASR module serves as input to the BERT-based classification system and the urgency detection model. Maintaining consistency in text normalization is critical to ensure that downstream models receive clean and semantically meaningful input. Therefore, post-processing steps include punctuation restoration and normalization of numerical expressions.